

UNIT – 5 LAB EXERSICE

NAME : Ranjitha. R

REG NO : 192421285

1. There are $3n$ piles of coins of varying size, you and your friends will take piles of coins as follows: In each step, you will choose any 3 piles of coins (not necessarily consecutive). Of your choice, Alice will pick the pile with the maximum number of coins. You will pick the next pile with the maximum number of coins. Your friend Bob will pick the last pile. Repeat until there are no more piles of coins. Given an array of integers piles where piles[i] is the number of coins in the ith pile. Return the maximum number of coins that you can have.

```
1- def maxCoins(piles):
2-     piles.sort()
3-     n = len(piles)
4-     ans = 0
5-     i = n - 2
6-
7-     for _ in range(n // 3):
8-         ans += piles[i]
9-         i -= 2
10-
11-     return ans
12-
13- piles = [2,4,1,2,7,8]
14- print(maxCoins(piles))
```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.984 Mb

Your Output

9

2. You are given a 0-indexed integer array coins, representing the values of the coins available, and an integer target. An integer x is obtainable if there exists a subsequence of coins that sums to x. Return the minimum number of coins of any value that need to be added to the array so that every integer in the range [1, target] is obtainable. A subsequence of an array is a new non-empty array that is formed from the original array by deleting some (possibly none) of the elements without disturbing the relative positions of the remaining elements.

```
1- def minPatches(coins, target):
2-     coins.sort()
3-     reach = 1
4-     i = 0
5-     count = 0
6-
7-     while reach <= target:
8-         if i < len(coins) and coins[i] <= reach:
9-             reach += coins[i]
10-            i += 1
11-         else:
12-             reach += reach
13-             count += 1
14-
15-     return count
16-
17- coins = [1,4,10]
18- target = 19
19- print(minPatches(coins, target))
```

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 8.884 Mb

Your Output

2

3. You are given an integer array jobs, where jobs[i] is the amount of time it takes to complete the ith job. There are k workers that you can assign jobs to. Each job should be assigned to exactly one worker. The working time of a worker is the sum of the time it takes to complete all jobs assigned to them. Your goal is to devise an optimal assignment such that the maximum working time of any worker is minimized. Return the minimum possible maximum working time of any assignment.

```

1 def minimumTime(jobs, k):
2     jobs.sort(reverse=True)
3     workers = [0] * k
4
5     def backtrack(i):
6         if i == len(jobs):
7             return True
8
9         for j in range(k):
10            if workers[j] + jobs[i] <= limit:
11                workers[j] += jobs[i]
12
13                if backtrack(i + 1):
14                    return True
15
16                workers[j] -= jobs[i]
17
18            if workers[j] == 0:
19                break
20
21        return False
22
23    low = max(jobs)
24    high = sum(jobs)
25
26    while low < high:
27        limit = (low + high) // 2
28        if backtrack(0):
29            high = limit
30        else:
31            low = limit + 1
32
33    return low
34
35 jobs = [1,2,4,7,8]
36 k = 2
37 print(minimumTime(jobs, k))

```

Enter Input here

If your code takes input, add it in the above box below

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.616 Mb

Your Output

15

4. We have n jobs, where every job is scheduled to be done from startTime[i] to endTime[i], obtaining a profit of profit[i]. You're given the startTime, endTime and profit arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range. If you choose a job that ends at time X you will be able to start another job that starts at time X.

```

1 def maxProfit(start, end, profit):
2     jobs = sorted(zip(start, end, profit))
3     n = len(jobs)
4     dp = [0] * (n + 1)
5
6     for i in range(1, n + 1):
7         s, e, p = jobs[i - 1]
8
9         j = i - 1
10        while j > 0 and jobs[j - 1][1] > s:
11            j -= 1
12
13        dp[i] = max(dp[i - 1], p + dp[j])
14
15    return dp[n]
16
17 start = [1,2,3,3]
18 end = [3,4,5,6]
19 profit = [50,10,40,70]
20
21 print(maxProfit(start, end, profit))

```

Enter Input here

If your code takes input, add it in the above box below

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.684 Mb

Your Output

120

5. Given a graph represented by an adjacency matrix, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to all other vertices in the graph. The graph is represented as an adjacency matrix where $graph[i][j]$ denote the weight of the edge from vertex i to vertex j . If there is no edge between vertices i and j , the value is Infinity (or a very large number).

```

1 def dijkstra(graph, source):
2     n = len(graph)
3     dist = [float('inf')] * n
4     visited = [False] * n
5     dist[source] = 0
6
7     for _ in range(n):
8         u = -1
9
10        for i in range(n):
11            if not visited[i] and (u == -1 or dist[i] < dist[u]):
12                u = i
13
14        visited[u] = True
15
16        for v in range(n):
17            if graph[u][v] != float('inf'):
18                dist[v] = min(dist[v], dist[u] + graph[u][v])
19
20    return dist
21
22 I = float('inf')
23
24 graph = [
25     [0,10,3,I,I],
26     [I,0,1,2,I],
27     [I,4,0,8,2],
28     [I,I,1,0,7],
29     [I,I,1,9,0]
30 ]
31
32 print(dijkstra(graph, 0))

```

Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 9 Mb

Your Output

[0, 7, 3, 9, 5]

6. Given a graph represented by an edge list, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to a target vertex. The graph is represented as a list of edges where each edge is a tuple (u, v, w) representing an edge from vertex u to vertex v with weight w .

```

1 import heapq
2
3 def dijkstra(n, edges, source, target):
4     graph = [[] for _ in range(n)]
5
6     for u, v, w in edges:
7         graph[u].append((v, w))
8
9     dist = [float('inf')] * n
10    dist[source] = 0
11    pq = [(0, source)]
12
13    while pq:
14        d, u = heapq.heappop(pq)
15
16        if u == target:
17            return d
18
19        if d > dist[u]:
20            continue
21
22        for v, w in graph[u]:
23            nd = d + w
24
25            if nd < dist[v]:
26                dist[v] = nd
27                heapq.heappush(pq, (nd, v))
28
29    return dist[target]
30
31 edges = [
32     (0,1,7),(0,2,9),(0,5,14),
33     (1,2,10),(1,3,15),(2,3,11),
34     (2,5,2),(3,4,6),(4,5,9)
35 ]
36
37 print(dijkstra(6, edges, 0, 4))

```

Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.144 Mb

Your Output

26

7. Given a set of characters and their corresponding frequencies, construct the Huffman Tree and generate the Huffman Codes for each character.

```

1 import heapq
2
3 def huffman(chars, freq):
4     heap = [[f, c, ""] for c, f in zip(chars, freq)]
5     heapq.heapify(heap)
6
7     while len(heap) > 1:
8         a = heapq.heappop(heap)
9         b = heapq.heappop(heap)
10
11         for x in a[1:]:
12             if isinstance(x, str):
13                 pass
14
15             heapq.heappush(heap, [a[0] + b[0], a, b])
16
17     codes = {}
18
19     def generate(node, code=""):
20         if isinstance(node[1], str):
21             codes[node[1]] = code
22         else:
23             generate(node[1], code + "0")
24             generate(node[2], code + "1")
25
26     generate(heap[0])
27     return codes
28
29 chars = ['a','b','c','d']
30 freq = [5,9,12,13]
31
32 print(huffman(chars, freq))

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.144 Mb

Your Output

```
{'a': '00', 'b': '01', 'c': '10', 'd': '11'}
```

8. Given a Huffman Tree and a Huffman encoded string, decode the string to get the original message.

```

1 def decode(codes, encoded):
2     result = ""
3     temp = ""
4
5     reverse = {v: k for k, v in codes.items()}
6
7     for bit in encoded:
8         temp += bit
9
10        if temp in reverse:
11            result += reverse[temp]
12            temp = ""
13
14    return result
15
16 codes = {
17     'a': '110',
18     'b': '10',
19     'c': '0',
20     'd': '111'
21 }
22
23 encoded = "1101100111110"
24
25 print(decode(codes, encoded))

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.964 Mb

Your Output

```
aacda
```

9. Given a list of item weights and the maximum capacity of a container, determine the maximum weight that can be loaded into the container using a greedy approach. The greedy approach should prioritize loading heavier items first until the container reaches its capacity.

```
1 def maxWeight(weights, capacity):
2     weights.sort(reverse=True)
3     total = 0
4
5     for w in weights:
6         if total + w <= capacity:
7             total += w
8
9     return total
10
11 weights = [10,20,30,40,50]
12 capacity = 60
13
14 print(maxWeight(weights, capacity))
```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:	Memory:
0.0200 secs	8.784 Mb

Your Output

60

10. Given a list of item weights and a maximum capacity for each container, determine the minimum number of containers required to load all items using a greedy approach. The greedy approach should prioritize loading items into the current container until it is full before moving to the next container.

```
1 def containers(weights, capacity):
2     count = 1
3     current = 0
4
5     for w in weights:
6         if current + w <= capacity:
7             current += w
8         else:
9             count += 1
10            current = w
11
12     return count
13
14 weights = [5,10,15,20,25,30,35]
15 capacity = 50
16
17 print(containers(weights, capacity))
```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	8.7 Mb

Your Output

4

11. Given a graph represented by an edge list, implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) and its total weight.

```

1 def kruskal(n, edges):
2     edges.sort(key=lambda x: x[2])
3     parent = list(range(n))
4     mst = []
5     total = 0
6
7     def find(x):
8         while parent[x] != x:
9             x = parent[x]
10        return x
11
12    for u, v, w in edges:
13        a = find(u)
14        b = find(v)
15
16        if a != b:
17            parent[a] = b
18            mst.append((u, v, w))
19            total += w
20
21        if len(mst) == n - 1:
22            break
23
24    return mst, total
25
26 edges = [
27     (0,1,10),
28     (0,2,6),
29     (0,3,5),
30     (1,3,15),
31     (2,3,4)
32 ]
33
34 mst, total = kruskal(4, edges)
35
36 print("Edges in MST:", mst)
37 print("Total weight:", total)

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.096 Mb

Your Output

Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
Total weight: 19

12. Given a graph with weights and a potential Minimum Spanning Tree (MST), verify if the given MST is unique. If it is not unique, provide another possible MST.

```

1 def mst_weight(n, edges, skip=-1):
2     edges = sorted(edges, key=lambda x: x[2])
3     parent = list(range(n))
4     total = 0
5     count = 0
6
7     def find(x):
8         if parent[x] != x:
9             parent[x] = find(parent[x])
10        return parent[x]
11
12    for i, (u, v, w) in enumerate(edges):
13        if i == skip:
14            continue
15
16        a = find(u)
17        b = find(v)
18
19        if a != b:
20            parent[a] = b
21            total += w
22            count += 1
23
24    return total if count == n - 1 else float('inf')
25
26
27 def checkUnique(n, edges):
28     edges = sorted(edges, key=lambda x: x[2])
29     best = mst_weight(n, edges)
30
31     for i in range(len(edges)):
32         if mst_weight(n, edges, i) == best:
33             return False
34
35     return True
36
37
38 edges = [
39     (0,1,1),
40     (0,2,1),
41     (1,2,1)
42 ]

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.128 Mb

Your Output

Is the MST unique? False